Canonical Physics and Conventions Reference for `cqed_sim`

Post-cleanup and protocol-alignment reference

March 17, 2026

This document is the source of truth for the reusable simulator code under `cqed_sim/`. It records the active physics conventions, the low-level state-preparation, measurement, and spectral primitives that remain inside the library, the cleaned package boundary versus `examples/`, and the tests that directly back the main conventions.

1 Overview of `cqed_sim`

`cqed_sim/` contains reusable simulator code rather than experiment-specific workflows:

- `core`: Hilbert-space conventions, frames, the universal subsystem-based cQED model layer, two-mode and three-mode compatibility wrappers, reusable state-preparation helpers, static-spectrum helpers, ideal gates, and manifold-frequency helpers.
- `pulses`, `sequence`, `sim`: pulse objects, envelopes, channel compilation, hardware distortion, solver paths, collapse operators, and extractors.
- `measurement`: reusable qubit readout abstractions, physical readout-chain models, and measurement utilities that remain backend-agnostic and library-oriented.
- `rl_control`, `system_id`: RL-facing environment wrappers, benchmark-task interfaces, domain-randomized parameter sampling, and calibration-informed prior hooks built on top of the same runtime stack.
- `calibration`, `observables`, `operators`, `tomo`, `io`, `plotting`, `unitary_synthesis`: reusable helpers that still belong to the library surface.

Study code, workflow code, paper-reproduction code, audits, and smoke tests remain under `examples/`.
The experiment-facing cleanup note is kept in `physics_and_conventions/experimental_protocol_alignment_no`

2 Global Modeling Assumptions

The reusable models in `cqed_sim` are effective cQED models rather than full circuit-quantization outputs. Unless explicitly stated otherwise, the simulator assumes:

- a reduced-mode description consisting of a transmon-like weakly anharmonic mode and one or more bosonic resonator modes,
- rotating-wave style effective interactions for the driven control terms,
- dispersive-model Hamiltonians when using the dispersive runtime classes,

- a Duffing-oscillator truncation for the transmon sector,
- finite Hilbert-space truncation for all modes,
- Markovian open-system dynamics in Lindblad form when dissipation is enabled,
- and piecewise-sampled or analytic control envelopes as implemented by the pulse layer.

Counter-rotating corrections, Bloch–Siegert shifts, full microwave-network electrodynamics, and full circuit-quantization renormalization are not included unless they are added explicitly through model extensions or effective fitted parameters.

3 Internal Hamiltonian Unit Convention

Throughout the simulator, internal Hamiltonian coefficients are represented in angular-frequency units. Equivalently, the code works with $H/\hbar$ rather than energy in joules. Thus:

- all static Hamiltonian coefficients are in rad/s,
- all rotating-frame frequencies and pulse carriers are in rad/s,
- all dissipation rates are in 1/s,
- all times are in seconds.

The evolution equations therefore use the standard convention

$$\dot{\rho} = -i[H, \rho] + \mathcal{L}(\rho),$$

where H denotes the internal Hamiltonian expressed in angular-frequency units. Whenever this document writes $H/\hbar$, it is emphasizing this internal convention. Whenever this document simply writes H, the same convention is still intended unless explicitly stated otherwise.

4 Bare, Dressed, and Measured Parameters

The symbols ω_q, ω_s, ω_c, and ω_r must be interpreted carefully:

- **Bare parameters** refer to the uncoupled circuit quantities that arise before dispersive reduction or effective-model fitting.
- **Runtime model parameters** in `cqed_sim` are generally *effective dressed parameters* of the chosen reduced Hamiltonian, unless a section explicitly says otherwise.
- **Measured frequencies** are experimentally extracted transition frequencies, usually obtained from spectroscopy, Ramsey fits, or energy differences inferred from the effective model.

Accordingly:

- the runtime Hamiltonian should be compared against measured transition frequencies and effective dressed parameters,
- bare circuit parameters belong primarily to translation, fitting, and analysis layers rather than to the main runtime dispersive Hamiltonian,
- and transition frequencies should be computed from energy differences within the chosen effective Hamiltonian rather than by reading raw bare parameter values directly.

5 Energy Offsets and Reporting Conventions

Absolute energies depend on frame choice and on the arbitrary choice of zero of energy. In this repository:

- identity shifts may be omitted from derived forms of Hamiltonians when they do not affect dynamics,
- rotating-frame Hamiltonians may produce negative excited-state energies,
- `EnergySpectrum.energies` are reported relative to the bare vacuum basis state of the selected model,
- transition frequencies and observable dynamics are determined by energy differences, not by the absolute value of the reported zero of energy.

Thus vacuum-referenced energy output is a reporting convention only; it does not change the Hamiltonian that was diagonalized.

6 State Preparation and Measurement Primitives

The reusable preparation and measurement helpers that remain in the library are implemented in:

- `cqed_sim/core/state_prep.py`: `StatePreparationSpec`, reusable subsystem state specs, and helpers such as `prepare_state(...)`.
- `cqed_sim/measurement/qubit.py`: `QubitMeasurementSpec` and `measure_qubit(...)`.
- `cqed_sim/measurement/readout_chain.py`: `ReadoutResonator`, `PurcellFilter`, `AmplifierChain`, and `ReadoutChain`.
- `cqed_sim/measurement/stochastic.py`: `ContinuousReadoutSpec`, `simulate_continuous_readout(...)`, and measurement-record integration helpers.
- `cqed_sim/measurement/strong_readout.py`: occupancy-activated strong-readout disturbance helpers for non-QND threshold studies.
- `cqed_sim/sim/noise.py`: reusable noise helpers such as `pure_dephasing_time_from_t1_t2(...)` used by library code and example workflows.

High-level experiment recipes and canned workflows no longer live under `cqed_sim/`. Guided notebook lessons now live under `tutorials/`, while repository-side workflow helpers and standalone scripts live under `examples/`. These library objects are not a second solver stack. They are reusable primitives that compose with `SequenceCompiler` plus `simulate_sequence(...)`. In particular:

- State preparation may return either a ket or a density matrix. If any subsystem specification is mixed, the joint initial state is promoted to a density matrix before tensoring.
- `measure_qubit(...)` returns exact probabilities by default. Sampled outcomes appear only when `shots` is set.

- When a confusion matrix is supplied, the code applies

$$\mathbf{p}_{\text{obs}} = M\mathbf{p}_{\text{latent}},$$

  with columns ordered as latent (g, e) and rows ordered as reported (g, e).

- If `QubitMeasurementSpec.readout_chain` is omitted, synthetic I/Q points are still generated from the lightweight Gaussian cluster model.

- If a `ReadoutChain` is provided, the measurement layer instead uses a resonator response, optional Purcell filter, amplifier noise, and mixer-style I/Q integration. The same API can also report γ_{meas} and a Purcell-limited T_1, and can optionally apply readout-induced dephasing plus Purcell relaxation as a reduced-state backaction model.

- `simulate_continuous_readout(...)` reuses the runtime Hamiltonian assembly and promotes one selected bosonic emission channel into the monitored SME path. Relaxation, thermal excitation, and dephasing remain unmonitored Lindblad terms.

- `integrate_measurement_record(...)` treats the final axis as time and preserves any leading monitored-operator or heterodyne-quadrature axes.

- `build_strong_readout_disturbance(...)` is an operational approximation: it estimates state-averaged readout occupancy from the linear dispersive response and converts large occupancy plus waveform slew into auxiliary multilevel transmon-drive envelopes. It is not a microscopic ionization model.

Direct tests: `tests/test_26_state_prep_and_measurement.py`, `tests/experiment/test_readout_chain.py`, `tests/experiment/test_stochastic_readout.py`.

7 RL Wrapper Conventions

The RL-facing stack in `cqed_sim/rl_control` is intentionally a wrapper around the existing runtime rather than a second simulator. It exposes a Gym-like `reset(...)` and `step(...)` interface, but it preserves the same unit, frame, pulse, tensor-ordering, and measurement conventions used throughout the rest of `cqed_sim`.

- All model coefficients remain in rad/s and all durations remain in s.

- The reduced RL regime is built from `DispersiveTransmonCavityModel`; the fuller pulse regime is built from `UniversalCQEDModel`. Neither path changes the transmon-first tensor ordering described later in this document.

- When `HybridSystemConfig.use_model_rotating_frame=True` and the configured `FrameSpec` is left at zero, the convenience layer uses the model's bare storage and qubit frequencies as the numerical rotating frame. This is a frame-selection convenience, not a new physical sign convention.

- Measurement-like RL observations are derived from `QubitMeasurementSpec` and therefore inherit the same confusion-matrix rule,

$$\mathbf{p}_{\text{obs}} = M\mathbf{p}_{\text{latent}},$$

  with latent columns ordered as (g, e) and reported rows ordered as (g, e).

- RL diagnostics may expose simulator-only quantities such as the full state, reduced states, Wigner data, or compiled pulses. Those are debugging and evaluation outputs only; they are not implied to be directly available to a laboratory policy.

Direct tests: `tests/test_38_rl_control_env.py`.

8 Optimal-Control Conventions

The direct optimal-control layer in `cqed_sim/optimal_control` is a wrapper around the existing model and pulse stack rather than a second control convention set. It therefore inherits the same Hamiltonian units, frame choices, tensor ordering, and drive-target semantics used elsewhere in the repository.

- All drift and control Hamiltonian coefficients remain in rad/s, and all slice durations remain in s.
- The current backend is a dense closed-system GRAPE solver on a piecewise-constant propagation grid. This is an implementation scope boundary, not a new physics convention.
- Parameter values, command waveforms, and physical waveforms are distinct layers. The explicit control path is $\theta \to u_{\mathrm{cmd}} \to u_{\mathrm{phys}} \to H(t; u_{\mathrm{phys}})$.
- `PiecewiseConstantParameterization` is the identity map from parameters to the propagation-grid command waveform, while `HeldSampleParameterization` stores coarser AWG-like samples and applies sample-and-hold onto that same propagation grid.
- Model-backed control problems are built from the existing `static_hamiltonian(frame)` and drive-operator helpers such as `drive_coupling_operators(...)`, `transmon_transition_operators(...)`, and `sideband_drive_operators(...)`. The optimizer does not introduce a separate tensor-ordering rule.
- The Hermitian optimizer quadratures are labeled as in-phase and quadrature controls, denoted here by $I(t)$ and $Q(t)$. When an optimized schedule is exported back into the standard pulse stack, the complex rotating-frame baseband coefficient is defined by

  $$c(t) = I(t) - iQ(t).$$

  This choice is what makes the exported `Pulse` objects reproduce the same effective Hermitian control Hamiltonian used during optimization under the repository's existing waveform sign convention.
- Leakage penalties are defined relative to a retained subspace in the same full Hilbert space used by the model and by the existing `unitary_synthesis` subspace machinery.
- Optional hardware maps such as first-order low-pass filters, radial I/Q limits, and boundary windows modify the physical waveform seen by GRAPE without changing the underlying Hamiltonian sign, frame, or tensor-ordering convention.
- The solver may internally rescale optimization variables to dimensionless bounded coordinates before calling the numerical optimizer. This is only a numerical conditioning step; the public problem definition and all reported physical amplitudes remain in rad/s.

- Simulator-backed replay through `evaluate_control_with_simulator(...)` or `ControlResult.evaluate_with` is an evaluation path, not a noisy-gradient optimizer. It can replay either the command waveform or the physical post-hardware waveform through the existing runtime with optional `NoiseSpec` Lindblad terms and reports the resulting probe-state fidelities using the same target states that defined the closed-system objective.

- For retained-subspace unitary objectives, the replay path also reports subspace leakage relative to the same retained subspace. This leakage report is therefore a runtime evaluation diagnostic, not the optimized cost unless an explicit closed-system `LeakagePenalty` was also included in the problem.

Direct tests: `tests/test_40_optimal_control_grape.py`, `tests/test_41_optimal_control_followup.py`, `tests/test_42_optimal_control_hardware_constraints.py`.

9 Physical Readout Chain

The physical readout-chain path models a driven dispersive resonator with optional Purcell filtering and additive amplifier noise. This section is an *effective measurement-model convention*, not the primary source of truth for the runtime Hamiltonian sign of the transmon-storage χ.

Effective readout-response notation

To avoid ambiguity with the Hamiltonian-level dispersive coefficients, it is useful to conceptually distinguish:

- a *qubit transition pull* per bosonic excitation, which appears in Hamiltonians such as $+\chi_s n_s n_q$,

- and an effective *resonator pointer-state separation*, which appears in the readout response model through different resonator frequencies for different qubit states.

In the current implementation, the same field name `chi` may appear in both contexts for compatibility, but they should be interpreted according to the layer being discussed:

- in the Hamiltonian sections below, χ means a conditional qubit-frequency shift,

- in the readout-chain section, the same symbol may parameterize the effective resonator pointer-state separation used by the response model.

Implemented effective model

- `ReadoutResonator` stores ω_r, κ, the qubit-resonator coupling g, the complex drive amplitude ϵ, and an optional effective resonator-pull parameter.

- The steady-state pointer amplitudes are

$$\alpha_{g/e}^{(\mathrm{ss})} = \frac{-i\epsilon}{\kappa_{\mathrm{eff}}/2 + i(\omega_{r,g/e} - \omega_d)},$$

  with $\omega_{r,g} = \omega_r$ and $\omega_{r,e} = \omega_r + \chi_{\mathrm{ro,eff}}$ in the effective pointer-state model.

- The implemented measurement-induced dephasing rate is

$$\gamma_{\text{meas}} = \frac{\kappa_{\text{eff}}}{2}|\alpha_e - \alpha_g|^2,$$

  which reproduces the expected small-pull scaling $\gamma_{\text{meas}} \propto \chi_{\text{ro,eff}}^2 \bar{n}/\kappa$.

- The Purcell filter follows the standard frequency-dependent linewidth picture:

$$\kappa_{\text{eff}}(\omega) = \frac{4J^2\kappa_f}{\kappa_f^2 + 4(\omega - \omega_f)^2},$$

  so the Purcell rate is evaluated as

$$\Gamma_P = (g/\Delta)^2 \kappa_{\text{eff}}(\omega_q).$$

- `AmplifierChain` applies linear gain and additive thermal noise before mixer-phase rotation and I/Q integration.

Direct tests: `tests/experiment/test_readout_chain.py`.

10 Sequential Sideband Reset Approximation

The sequential storage-reset workflow lives in `examples/workflows/sequential_sideband_reset.py`. It is an example-level orchestration built on the explicit three-mode dispersive runtime rather than a second simulator stack. The modeled protocol is

$$|g, 0_r, n_s\rangle \overset{\text{storage red sideband}}{\leftrightarrow} |f, 0_r, n_s - 1\rangle \overset{\text{readout red sideband}}{\leftrightarrow} |g, 1_r, n_s - 1\rangle \xrightarrow{\kappa_r} |g, 0_r, n_s - 1\rangle.$$

For initial $|e, 0_r, n_s\rangle$ population, the helper first applies an explicit $e \to f$ transmon pulse and then performs the readout-sideband dump before entering the photon-removal loop.
This is a controlled effective-model approximation. The readout-assisted evacuation stage is represented as a driven effective $g \leftrightarrow f$ red sideband on the readout mode followed by explicit Lindblad readout loss, not as an autonomous always-on microscopic transmon–readout exchange Hamiltonian.

11 Static Spectrum Convention

The reusable spectral helper in `cqed_sim/core/spectrum.py` diagonalizes the static Hamiltonian selected by the caller and returns a structured `EnergySpectrum` result with both raw eigenvalues and vacuum-referenced energies.

- The diagonalized operator is exactly `model.static_hamiltonian(frame)` for the chosen `FrameSpec`.
- Reported `EnergySpectrum.energies` are shifted by subtracting the energy of the bare vacuum basis state.
- The bare vacuum means every subsystem is in level 0: $|g, 0\rangle$ for the two-mode model, $|g, 0_s, 0_r\rangle$ for the three-mode model, and $|0\rangle$ for cavity-only models.
- This shift changes only the reported zero of energy. It does not change which Hamiltonian was diagonalized or which transition frequencies follow from energy differences.
- In the lab frame, the low-lying spectrum is the most directly interpretable dressed ladder. In rotating frames, excited levels can appear at negative energy even though the vacuum remains the zero-reference state.

Direct tests: `tests/analysis/test_spectrum.py`.

12 Hilbert Space Structure

Two-mode runtime

- Runtime ordering is transmon first, storage second:

$$|q, n\rangle = |q\rangle_{\text{transmon}} \otimes |n\rangle_{\text{storage}}.$$

- The flattened basis index is

$$\texttt{index}(q, n) = q\, n_{\text{cav}} + n,$$

  implemented in `cqed_sim/core/conventions.py`.

- Computational basis labels are

$$|g\rangle \equiv |0\rangle, \qquad |e\rangle \equiv |1\rangle, \qquad |f\rangle \equiv |2\rangle,$$

  and more generally transmon level j corresponds to Duffing ladder state $|j\rangle$.

Three-mode runtime

- The three-mode extension uses

$$|q, n_s, n_r\rangle = |q\rangle_{\text{transmon}} \otimes |n_s\rangle_{\text{storage}} \otimes |n_r\rangle_{\text{readout}}.$$

- The flattened basis index is

$$\texttt{index}(q, n_s, n_r) = ((q\, n_{\text{storage}}) + n_s)\, n_{\text{readout}} + n_r.$$

- This ordering is implemented in `cqed_sim/core/conventions.py` and `cqed_sim/core/readout_model.py`, and new extractors assume subsystem 0 is the transmon, subsystem 1 is storage, subsystem 2 is readout.

Universal model layer

- The reusable model core lives in `cqed_sim/core/universal_model.py`.
- `UniversalCQEDModel` accepts an optional `TransmonModeSpec`, a sequence of `BosonicModeSpec` entries, explicit `DispersiveCouplingSpec` entries, and generic `CrossKerrSpec`, `SelfKerrSpec`, and `ExchangeSpec` terms.
- Tensor ordering remains explicit: the transmon is first when present, followed by bosonic modes in declaration order.
- `DispersiveTransmonCavityModel` and `DispersiveReadoutTransmonStorageModel` preserve their historical public constructors and delegate Hamiltonian and operator logic to the shared universal layer.

Synthesis subspace

The unitary-synthesis layer retains the same full-space transmon-first ordering, but restricted subspace bases are explicitly reordered in `cqed_sim/unitary_synthesis/subspace.py`; for example, a transmon-cavity block is ordered as

$$|g,0\rangle,\ |e,0\rangle,\ |g,1\rangle,\ |e,1\rangle,\ \ldots$$

Direct tests: `tests/test_25_tensor_product_convention.py`, `tests/test_27_three_mode_model.py`, `tests/unitary_synthesis/test_subspace.py`.
The flexible synthesis API now also permits matrix-defined primitives and model-backed waveform-defined primitives. The waveform-defined path does not introduce a second pulse convention: it reuses the same `Pulse`, `SequenceCompiler`, and runtime simulation stack as the rest of `cqed_sim`. Therefore it inherits the same carrier-sign rule discussed later in this document,

$$\texttt{Pulse.carrier} = -\omega_{\text{transition}}(\text{frame}).$$

An explicit cavity-only logical block-phase layer is represented in the same transmon-first tensor convention as

$$I_{\text{q}} \otimes \text{diag}(e^{i\phi_{n_0}}, e^{i\phi_{n_1}}, \ldots),$$

with identity on omitted cavity levels. In the restricted synthesis basis this appears as equal phase factors on the paired two-dimensional blocks $|g,n\rangle, |e,n\rangle$. Gauge-fixed logical block-phase diagnostics are therefore defined blockwise relative to the first selected logical block rather than relative to the absolute full-space phase. Unitary targets correspond to closed-system effective propagators, while state-mapping targets are the appropriate synthesis-side abstraction when the waveform simulation includes dissipation or other open-system effects. Phase-2 synthesis extends that rule in a controlled way: when a user supplies a unitary target together with open-system waveform simulation, the optimization objective is evaluated through propagated probe states rather than by extracting an exact effective unitary from Lindblad evolution. This keeps the physical meaning aligned with the runtime solver while still allowing unitary-like target specification in noisy synthesis workflows. Phase-2 robustness features also sample uncertain Hamiltonian parameters by cloning the user model and re-evaluating the same synthesis objective under those sampled parameter sets; they do not introduce an alternate Hamiltonian sign convention or basis ordering.

13 Holographic Quantum-Algorithm Conventions

The reusable holographic algorithms package in `cqed_sim/quantum_algorithms/holographic_sim/` is intentionally system-agnostic. Its core factorization is

$$|\sigma\rangle_{\text{physical}} \otimes |b\rangle_{\text{bond}},$$

with the *physical register first* and the persistent *bond register second*. This ordering is independent of the cQED-specific transmon–bosonic tensor ordering used by the main simulator runtime.
For a dense joint unitary U acting on $\mathcal{H}_{\text{physical}} \otimes \mathcal{H}_{\text{bond}}$, the implemented holographic channel prepares a fixed physical reference state $|0\rangle_{\text{physical}}$ by default and uses the standard Kraus map

$$\rho_{\text{bond}} \mapsto \sum_{\sigma} K_\sigma \rho_{\text{bond}} K_\sigma^\dagger, \qquad K_\sigma = \langle\sigma|U|0\rangle.$$

When a right-canonical MPS tensor is used instead, the implementation stores the MPS matrices V_σ together with the standard channel-orientation Kraus operators

$$K_\sigma = V_\sigma^\dagger.$$

This keeps the user-facing sampler aligned with standard quantum-channel notation while remaining consistent with the transfer-matrix orientation used in the holographic report.
The public helper `right_canonical_tensor_to_stinespring_unitary(...)` completes the same right-canonical tensor into a dense unitary whose first χ columns match the repository's default physical-input convention $|0\rangle_{\text{physical}}$, so it is consistent with `HolographicChannel.from_unitary(...)` and the legacy `holographicSim.py` execution path.
The public finite-step interface now also makes subsystem embedding explicit. When a step unitary is declared as physical-only, the implemented convention is

$$U_{\text{joint}} = U_{\text{physical}} \otimes I_\chi.$$

When a step unitary is declared as bond-only, the implemented convention is

$$U_{\text{joint}} = I_d \otimes U_{\text{bond}}.$$

Finite non-translation-invariant step programs are represented by `HolographicChannelSequence`. All steps in such a sequence share one physical dimension d and one bond dimension χ, but the unitary or Kraus data may differ from step to step.
The package uses `burn_in` to mean repeated application of the same holographic channel before observable insertions begin. This is the repository's convention for approaching bulk or steady-state behavior in the bond register. Observable insertions always refer to measurements on the physical register. Because this definition assumes one repeated channel, nonzero `burn_in` is intentionally supported for single-channel workflows and disallowed for finite explicit step sequences whose full step list is already specified.
The package also supports optional bond-only noise between holographic steps via `BondNoiseChannel`. When the convenience constructor `BondNoiseChannel.dephasing(...)` is used, the repository convention is computational-basis bond dephasing,

$$\mathcal{D}_p(\rho) = (1-p)\rho + p\sum_{b=0}^{\chi-1} |b\rangle\langle b|\rho|b\rangle\langle b|,$$

so diagonal bond populations are unchanged while off-diagonal bond coherences are multiplied by $1-p$. If the constructor is parameterized by `duration` and `tphi`, then the implemented convention is

$$p = 1 - e^{-t/T_\phi}.$$

When `BondNoiseChannel.amplitude_damping(...)` is used, the repository convention is computational-basis relaxation toward a designated target basis state $|b_{\text{target}}\rangle$,

$$\mathcal{A}_p(\rho) = K_0\rho K_0^\dagger + \sum_{b\neq b_{\text{target}}} K_b\rho K_b^\dagger,$$

with

$$K_0 = |b_{\text{target}}\rangle\langle b_{\text{target}}| + \sqrt{1-p}\sum_{b\neq b_{\text{target}}} |b\rangle\langle b|, \qquad K_b = \sqrt{p}\,|b_{\text{target}}\rangle\langle b|.$$

For bond dimension $\chi = 2$ and $b_{\text{target}} = 0$, this reduces to the standard qubit amplitude-damping channel. If the constructor is parameterized by `duration` and `t1`, then the implemented convention is

$$p = 1 - e^{-t/T_1}.$$

When `BondNoiseChannel.depolarizing(...)` is used, the implemented bond-space convention is isotropic depolarization toward the maximally mixed state,

$$\mathcal{P}_p(\rho) = (1-p)\rho + p\,\frac{I_\chi}{\chi}.$$

Because this noise acts only on the bond register and is trace preserving, applying it after physical conditioning is equivalent to applying it immediately before the next holographic step.
Direct tests: `tests/quantum_algorithms/test_holographic_sim.py`.

14 Operator Conventions

The simulator uses the standard bosonic normalization

$$[a, a^\dagger] = 1, \qquad n = a^\dagger a,$$

and for the Duffing transmon ladder before truncation,

$$[b, b^\dagger] = 1, \qquad n_q = b^\dagger b.$$

- The simulator uses QuTiP Pauli matrices without sign changes. Therefore

$$\sigma_z|g\rangle = +|g\rangle, \qquad \sigma_z|e\rangle = -|e\rangle.$$

- The core Hamiltonians use the excitation number operator $n_q = b^\dagger b$, not $\sigma_z/2$. For a strict two-level truncation,

$$n_q = |e\rangle\langle e| = \frac{I - \sigma_z}{2}.$$

- The shared operator builder lives in `cqed_sim/core/universal_model.py`; the specialized wrapper models keep the same user-facing operator semantics while delegating tensor lifting and caching to that shared path.

- Two-mode operators are embedded as

$$a = I_q \otimes a_{\text{storage}}, \qquad b = b_{\text{transmon}} \otimes I_s.$$

- Three-mode operators are embedded as

$$b = b_{\text{transmon}} \otimes I_s \otimes I_r, \quad a_s = I_q \otimes a_{\text{storage}} \otimes I_r, \quad a_r = I_q \otimes I_s \otimes a_{\text{readout}}.$$

- The generic runtime resolves drive operators through `model.drive_coupling_operators()`, so the solver no longer hard-codes a transmon-plus-one-mode assumption.

Direct tests: `tests/test_16_ideal_primitives_and_extractors.py`, `tests/test_20_gaussian_iq_convention` `tests/test_25_tensor_product_convention.py`, `tests/test_27_three_mode_model.py`.

15 Hamiltonian Definitions

Two-mode runtime Hamiltonian

The reusable two-mode Hamiltonian exposed by `cqed_sim/core/model.py` and assembled by the shared universal model layer is

$$H_0 = \delta_s n_s + \delta_q n_q + \frac{\alpha}{2} b^{\dagger 2} b^2 + K(n_s) + \Xi(n_s) n_q,$$

with

$$K(n_s) = \frac{\texttt{kerr}}{2} n_s(n_s - 1) + \cdots, \qquad \Xi(n_s) = \texttt{chi}\, n_s + \texttt{chi2}\, n_s(n_s-1) + \texttt{chi3}\, n_s(n_s-1)(n_s-2) + \cdots.$$

`cqed_sim/core/frequencies.py` makes the intended transition law explicit:

$$\omega_{ge}(n) = \omega_{ge}(0) + \Xi(n).$$

This is the canonical runtime sign convention:

- negative χ lowers the transmon $g \leftrightarrow e$ transition frequency as storage photon number increases,
- positive χ raises that transition frequency.

Textbook-to-code translation

With the active runtime convention,

$$n_q = \frac{I - \sigma_z}{2}.$$

Therefore

$$\delta_q n_q = \frac{\delta_q}{2} I - \frac{\delta_q}{2} \sigma_z, \qquad \Xi(n_s) n_q = \frac{\Xi(n_s)}{2} I - \frac{\Xi(n_s)}{2} \sigma_z.$$

Up to identity and storage-only shifts,

$$H_{q,\text{runtime}} \doteq -\frac{1}{2}\big(\delta_q + \Xi(n_s)\big)\sigma_z.$$

If one instead uses the common textbook sign

$$\sigma_z^{\text{text}} = |e\rangle\langle e| - |g\rangle\langle g| = -\sigma_z,$$

then

$$H_{q,\text{runtime}} \doteq \frac{1}{2}\big(\delta_q + \Xi(n_s)\big)\sigma_z^{\text{text}}.$$

Hence the projector-based runtime convention is fully compatible with common σ_z-based textbook forms after accounting for the sign choice of σ_z.

Comparison with common literature dispersive forms

Many cQED references begin from a dispersive Hamiltonian of the form

$$H_{\text{disp}} \sim \omega_s a^\dagger a + \frac{\omega_q}{2}\sigma_z + \chi a^\dagger a\, \sigma_z,$$

up to conventions, offsets, and sign choices for σ_z. In this repository, the canonical two-level projector form is instead

$$H_{\text{disp,repo}} \sim \delta_s n_s + \delta_q n_q + \chi n_s n_q.$$

The main advantage of the repository form is operational clarity:

$$\omega_{ge}(n) = \omega_{ge}(0) + \chi n.$$

This makes the meaning of the sign of χ explicit without requiring the reader to track a separate σ_z convention.

Transmon anharmonicity convention

The transmon nonlinearity is written as

$$\frac{\alpha}{2} b^{\dagger 2} b^2.$$

For a physical transmon, α is usually negative. Thus the adjacent ladder spacing decreases with excitation number, so the $e \to f$ transition is typically lower in frequency than the $g \to e$ transition.

Comparison with the synthesis drift convention

`cqed_sim/unitary_synthesis/sequence.py` uses the same projector-based dispersive and Kerr semantics as the runtime model:

$$E_{g,n} = \delta_s n + K(n), \qquad E_{e,n} = \delta_s n + \delta_q + \Xi_{\text{synth}}(n) + K(n).$$

Its transition shift is therefore

$$E_{e,n} - E_{g,n} = \delta_q + \Xi_{\text{synth}}(n).$$

Matching the runtime shift $\delta_q + \Xi_{\text{runtime}}(n)$ gives

$$\Xi_{\text{synth}}(n) = \Xi_{\text{runtime}}(n),$$

so componentwise

$$\chi_{\text{synth}} = \chi_{\text{runtime}}, \qquad \chi_{2,\text{synth}} = \chi_{2,\text{runtime}}, \qquad \chi_{3,\text{synth}} = \chi_{3,\text{runtime}}.$$

Likewise the cavity self-Kerr contribution is shared directly:

$$K_{\text{synth}}(n) = K_{\text{runtime}}(n).$$

The compatibility fields `synthesis_chi` and `synthesis_chi_2` in `cqed_sim/analysis/parameter_translation.py` are therefore aligned copies rather than sign-converted quantities.

Kerr free-evolution workflow presets

The example workflow in `examples/workflows/kerr_free_evolution.py` builds a two-mode storage–transmon model from human-readable frequencies in Hz and converts them to the runtime's angular-frequency units with `physics_and_conventions.conventions.to_internal_units()`. Its default frame is

$$\omega_{s,\text{frame}} = \omega_s, \qquad \omega_{q,\text{frame}} = \omega_q,$$

so the undriven evolution isolates the nonlinear Kerr and dispersive terms after removing the bare storage and transmon carrier rotations.
The Hamiltonian used by this workflow is exactly the two-mode runtime Hamiltonian evaluated in that rotating frame:

$$\frac{H_{\text{free}}}{\hbar} = \frac{\alpha}{2} b^{\dagger 2} b^2 + \frac{K}{2} n_s(n_s - 1) + \frac{K'}{6} n_s(n_s - 1)(n_s - 2) + \left[\chi n_s + \chi' n_s(n_s - 1) + \chi'' n_s(n_s - 1)(n_s - 2)\right] n_q.$$

When the transmon is initialized in $|g\rangle$, the reduced storage state undergoes pure self-Kerr evolution; the dispersive coefficients remain part of the joint Hamiltonian and matter whenever a different transmon preparation is used.

Three-mode storage–transmon–readout Hamiltonian

The three-mode model exposed by `cqed_sim/core/readout_model.py` and assembled by the shared universal model layer implements

$$\frac{H}{\hbar} = \delta_s n_s + \delta_r n_r + \delta_q n_q + \frac{\alpha}{2} b^{\dagger 2} b^2 + \chi_s n_s n_q + \chi_r n_r n_q + \chi_{sr} n_s n_r + \frac{K_s}{2} n_s(n_s - 1) + \frac{K_r}{2} n_r(n_r - 1).$$

This is intentionally kept in the same runtime convention style as the two-mode path:

- $n_q = b^\dagger b$ is the transmon excitation projector/number operator,
- positive χ_s or χ_r raises the transmon transition frequency as storage or readout occupancy increases,
- positive χ_{sr} raises the storage frequency with readout occupancy and raises the readout frequency with storage occupancy,
- positive K_s or K_r raises the adjacent bosonic spacing by $+Kn$.

The model also exposes analytic helpers for basis energies and for the transmon, storage, and readout transition frequencies.

Optional nonlinear and exchange terms

`cqed_sim/sim/couplings.py` provides explicit builder functions for

$$\chi_{ab} a^\dagger a\, b^\dagger b, \qquad +\frac{K}{2} a^{\dagger 2} a^2, \qquad J(a^\dagger b + a b^\dagger),$$

and the model classes accept `CrossKerrSpec`, `SelfKerrSpec`, and `ExchangeSpec` entries that are injected into the static Hamiltonian assembly. This extends the existing dispersive models without replacing their original API.

Higher-order nonlinear coefficients

Higher-order χ and Kerr coefficients are stored in falling-factorial form, not ordinary monomial form. For example:

$$\chi_2\, n(n-1), \qquad \chi_3\, n(n-1)(n-2),$$

rather than coefficients multiplying n^2 or n^3 directly. This is a repository-specific convention and must be preserved consistently in translation, fitting, and documentation.

Canonical parameter-definition table

Name	Code-level meaning	Algebra used	Caveat
`chi`	two-mode first-order dispersive pull	$+\texttt{chi}\, n_s n_q$, $\omega_{ge}(n) = \omega_{ge}(0) + \chi n$	negative values lower ω_{ge} with storage photons
`chi2`	second-order pull, exposed by `chi_higher[0]`	$+\chi_2 n_s(n_s-1)n_q$	falling-factorial coefficient
`chi3`	third-order pull, exposed by `chi_higher[1]`	$+\chi_3 n_s(n_s-1)(n_s-2)n_q$	falling-factorial coefficient
`chi_s`	storage–transmon dispersive pull in the three-mode model	$+\chi_s n_s n_q$	negative values lower ω_{ge} with storage photons
`chi_r`	readout–transmon dispersive pull in the three-mode model	$+\chi_r n_r n_q$	negative values lower ω_{ge} with readout photons
`chi_sr`	storage–readout cross-Kerr	$+\chi_{sr} n_s n_r$	positive values raise both bosonic mode frequencies with occupancy of the other mode
`kerr`	two-mode storage self-Kerr coefficient	$+\texttt{kerr}\, n_s(n_s-1)/2$	positive values raise the adjacent storage spacing by $+\texttt{kerr}\, n$
higher Kerr	tuple `kerr_higher`	$K_m n_s^{\underline{m}}/m!$	falling-factorial hierarchy
`kerr_s`	three-mode storage self-Kerr	$+\texttt{kerr_s}\, n_s(n_s-1)/2$	same sign convention as `kerr`
`kerr_r`	three-mode readout self-Kerr	$+\texttt{kerr_r}\, n_r(n_r-1)/2$	same sign convention as `kerr`
`alpha`	transmon Duffing anharmonicity	$\texttt{alpha}\, b^{\dagger 2} b^2/2$	typically negative for a physical transmon
σ_z	active runtime Pauli-Z	$\lvert g\rangle\langle g\rvert - \lvert e\rangle\langle e\rvert = I - 2n_q$	QuTiP standard sign
n_q	transmon excitation operator	$b^\dagger b$; for two levels $n_q = \lvert e\rangle\langle e\rvert$	repository uses projector/number notation rather than $\sigma_z/2$
δ_q	transmon residual detuning	$\omega_q - \omega_q^{\text{frame}}$	only defined after choosing a frame
δ_s	storage residual detuning	$\omega_s - \omega_s^{\text{frame}}$	same frame caveat
δ_r	readout residual detuning	$\omega_r - \omega_r^{\text{frame}}$	only used in the three-mode path

Standard-compatible versus repository-specific

- Standard-compatible: transmon-first tensor ordering, QuTiP Pauli signs, rad/s and seconds internally, Duffing/Kerr terms, Lindblad dynamics, and the canonical rule that negative χ lowers the transmon transition with bosonic occupancy while positive χ raises it.
- Repository-specific: writing the transmon term in terms of n_q, storing higher-order pulls and Kerr terms in falling-factorial form, and using complex envelopes with $e^{+i(\omega t+\phi)}$.
- Needs caution: raw `Pulse.carrier` remains the negative of the rotating-frame transition frequency, so user-facing spectroscopy axes should use `carrier_for_transition_frequency(...)` or `transition_frequency_from_carrier(...)` rather than raw carriers.

16 Frequency and Time Units

- Internal Hamiltonian coefficients and rotating-frame frequencies are in rad/s.
- Pulse durations, timeline steps, and solver `max_step` values are in seconds.
- The repository centralizes conversion helpers in `physics_and_conventions/conventions.py`, including `to_internal_units(...)` and `from_internal_units(...)`.
- Example-side configuration files may still carry `_hz`, `_ns`, or `_1_per_s` field names, but those values are converted before constructing library models or noise objects.

Direct tests: `tests/test_05_detuning_and_frames.py`, `tests/test_10_chi_convention.py`, `tests/test_13_ha` `tests/test_27_three_mode_model.py`.

17 Parameter Translation, Calibration Targets, and Backends

- `cqed_sim/analysis/parameter_translation.py` provides `from_transmon_params(...)` and `from_measured(...)`. Bare transmon parameters use the usual large-E_J/E_C expansion $\omega_{01} \approx \sqrt{8E_JE_C} - E_C$ and $\alpha \approx -E_C$, while dressed χ and χ_2 are extracted by matching a low-dimensional Duffing-plus-resonator diagonalization to the runtime form $\omega_{ge}(n) = \omega_q + \chi n + \chi_2 n(n-1)$.
- `cqed_sim/calibration_targets/` provides reusable spectroscopy, Rabi, Ramsey, T_1, $T_{2,\mathrm{echo}}$, and DRAG-tuning surrogate targets that return fitted parameters, uncertainties, and raw simulated data.
- `cqed_sim/backends/` provides `NumPyBackend` and `JaxBackend` plus a dense piecewise-constant solver path in `cqed_sim/sim/solver.py`. The default runtime still uses QuTiP; the backend path is opt-in through `SimulationConfig.backend`.

Direct tests: `tests/analysis/test_parameter_translation.py`, `tests/calibration_targets/test_targets.p` `tests/sim/test_backends.py`.

18 Rotating Frame Conventions

- Frames are represented by `FrameSpec(omega_c_frame, omega_q_frame, omega_r_frame)` in `cqed_sim/core/frame.py`.
- The legacy field `omega_c_frame` remains the storage/cavity frame frequency. The readout model adds the alias property `omega_s_frame` for clarity.
- The two-mode path uses
$$\delta_s = \omega_s - \omega_s^{\text{frame}}, \qquad \delta_q = \omega_q - \omega_q^{\text{frame}}.$$
- The three-mode path uses
$$\delta_s = \omega_s - \omega_s^{\text{frame}}, \qquad \delta_r = \omega_r - \omega_r^{\text{frame}}, \qquad \delta_q = \omega_q - \omega_q^{\text{frame}}.$$
- `manifold_transition_frequency(...)` still refers only to the two-mode path; the three-mode model instead exposes `qubit_transition_frequency(...)`, `storage_transition_frequency(...)`, and `readout_transition_frequency(...)`.

Direct tests: `tests/test_05_detuning_and_frames.py`, `tests/test_21_qubox_convention_reconciliation.py` `tests/test_27_three_mode_model.py`.

19 Drive Hamiltonians

- `cqed_sim/pulses/pulse.py` samples analytic or discrete envelopes as
$$\epsilon(t) = \texttt{amp}\,\texttt{envelope}(t)\, e^{i(\texttt{carrier}t+\texttt{phase})}.$$
- `cqed_sim/sim/runner.py` inserts $\epsilon(t)$ and $\epsilon^*(t)$ as raising and lowering couplings obtained from the model, rather than from a hard-coded target list.
- Because the runtime drive term is
$$\epsilon(t)\, O^\dagger + \epsilon^*(t)\, O,$$
a rotating-frame transition at frequency $\omega_{\text{transition}}$ is resonantly addressed by
$$\texttt{carrier} = -\omega_{\text{transition}}.$$
The public helpers `carrier_for_transition_frequency(...)` and `transition_frequency_from_carrier(..` make that mapping explicit.
- User-facing spectroscopy examples therefore plot physical transition detuning on the x-axis and convert that value to the internal raw carrier with `carrier_for_transition_frequency(...)` before simulation.
- The two-mode path supports `"cavity"` or `"storage"`, `"qubit"`, and the legacy `"sideband"` target, plus the structured targets `TransmonTransitionDriveSpec(lower_level, upper_level)` and `SidebandDriveSpec(mode, lower_level, upper_level, sideband)`.
- The three-mode path supports `"storage"`, `"readout"`, `"qubit"`, and `"transmon"` targets, plus the same structured transition and sideband targets.
- The effective three-mode drive Hamiltonian is
$$\frac{H_d}{\hbar} = \epsilon_s(t)a_s^\dagger + \epsilon_s^*(t)a_s + \epsilon_q(t)b^\dagger + \epsilon_q^*(t)b + \epsilon_r(t)a_r^\dagger + \epsilon_r^*(t)a_r.$$

Carrier-sign warning

Warning: raw `Pulse.carrier` is an internal rotating-frame quantity. It is not the same thing as a user-facing physical detuning axis. External spectroscopy and plotting code should convert through `carrier_for_transition_frequency(...)` or `transition_frequency_from_carrier(...)` rather than interpreting the raw sign directly.
Direct tests: `tests/test_20_gaussian_iq_convention.py`, `tests/test_23_sqr_additive_amplitude_correcti` `tests/test_28_three_mode_drives_and_noise.py`, `tests/test_31_multilevel_sideband_extension.py`.

Floquet analysis conventions

The public Floquet module in `cqed_sim/floquet` assumes a strictly periodic closed-system Hamiltonian

$$H(t+T) = H(t), \qquad \Omega = \frac{2\pi}{T}.$$

Its primary propagator-based route uses the one-period evolution operator

$$U(T)|\phi_\alpha(0)\rangle = e^{-i\varepsilon_\alpha T}|\phi_\alpha(0)\rangle,$$

so the quasienergies ε_α are defined only modulo the drive angular frequency Ω. The optional Sambe-space helper uses the truncated harmonic-space convention

$$\langle m|H_F|n\rangle = H^{(m-n)} + n\Omega\,\delta_{mn}.$$

- Floquet problems reuse the existing model-layer `static_hamiltonian(frame=...)` path and the same structured drive targets used by the pulse runtime.
- A target-based `PeriodicDriveTerm` defaults to the Hermitian in-phase quadrature built from the target's raising and lowering operators.
- Explicit operator-based Floquet terms are the intended path for periodic modulation of transmon frequency, cavity frequency, or dispersive parameters.
- Multi-tone problems are only strictly Floquet when the tones are commensurate with the declared common period.
- The current public Floquet API is closed-system. QuTiP provides lower-level Floquet-Markov primitives, but they are not yet surfaced through the public `cqed_sim` API.

Direct tests: `tests/test_58_floquet.py`.

Effective multilevel sideband model

The reusable sideband path added in `cqed_sim/core/drive_targets.py` and `cqed_sim/pulses/builders.py` is an effective rotating-wave Hamiltonian, not a microscopic flux-drive derivation. For a red sideband on a bosonic mode a_m and a selected transmon transition $|\ell\rangle \leftrightarrow |u\rangle$, the implemented operator pair is

$$\frac{H_{\mathrm{sb}}}{\hbar} = \epsilon(t)\,|u\rangle\langle\ell|\,a_m + \epsilon^*(t)\,a_m^\dagger|\ell\rangle\langle u|,$$

with an analogous blue-sideband variant that swaps a_m and $a_m^\dagger$. The public `SidebandDriveSpec` selects the bosonic mode, the ancilla ladder transition, and the red/blue branch. The helper `effective_sideband_rabi_frequency(g, δ)` uses the standard effective-model rule

$$\Omega_{\text{eff}} = \sqrt{(2g)^2 + \delta^2}.$$

This path is intended for fast-control and synchronization studies where the effective manifold dynamics are the quantity of interest.

Kerr-sign notebook diagnosis

The paper-comparison notebook `test_against_papers/prl115_137002_bosonic_controls.ipynb` was audited against the runtime implementation. The core simulator and the documented physics convention both use

$$\frac{H_K}{\hbar} = +\frac{K}{2} a^{\dagger 2} a^2.$$

For the measured parameter set used in the notebook, the runtime evolution matches the documented negative-K behavior. The apparent sign flip comes from interpreting the rotating-frame coherent-state phase trajectory, not from a flipped self-Kerr term in the Hamiltonian builder. This is guarded by `tests/test_32_kerr_sign_notebook_regression.py` and exposed through `cqed_sim.experiment.verify_kerr_sign(...)`.

20 Dissipation and Lindblad Terms

- `NoiseSpec` in `cqed_sim/sim/noise.py` supports `t1`, `transmon_t1`, `tphi`, `tphi_storage`, `tphi_readout`, `kappa`, `nth`, `kappa_storage`, `kappa_readout`, `nth_storage`, and `nth_readout`.
- The implemented convention is

$$\gamma_1 = 1/T_1, \qquad \gamma_\phi = 1/(2T_\phi).$$

- If `transmon_t1=(T1_ge, T1_fe, ...)` is supplied, the collapse-operator builder emits explicit ladder decays

$$\sqrt{1/T_{1,j}}\, |j-1\rangle\langle j|.$$

  If it is not supplied, the older aggregate $\sqrt{\gamma_1}\, b$ path remains available through `t1`.
- Two-level dephasing uses $\sqrt{\gamma_\phi}\, \sigma_z$ with $\sigma_z = I - 2n_q$. Multi-level transmon dephasing uses $\sqrt{\gamma_\phi}\, n_q$.
- Storage and readout pure dephasing use $\sqrt{1/T_{\phi,s}}\, n_s$ and $\sqrt{1/T_{\phi,r}}\, n_r$, respectively.
- These prefactors are chosen so that the relevant off-diagonal coherence decays as e^{-t/T_ϕ} for the intended subsystem convention.
- In the two-mode path, `kappa` and `nth` still drive storage loss and thermal excitation.
- In the three-mode path, storage and readout can decay independently via `kappa_storage` and `kappa_readout`, with separate thermal occupations.

- The helper `pure_dephasing_time_from_t1_t2(...)` uses

$$\frac{1}{T_2} = \frac{1}{2T_1} + \frac{1}{T_\phi}$$

  and clamps the inferred $1/T_\phi$ rate to zero. Because the helper returns a dephasing time rather than a rate, that zero-extra-dephasing case is encoded by returning `None`.

- `simulate_sequence(...)` uses `mesolve` when collapse operators are present or when the initial state is already a density matrix; otherwise it uses `sesolve`.

Direct tests: `tests/test_14_dissipation.py`, `tests/test_18_noise_and_sideband.py`, `tests/test_28_three_` `tests/test_31_multilevel_sideband_extension.py`.

21 State Representation

- Runtime inputs may be kets or density matrices.
- The result wrapper stores the final state, optional state history, expectation traces, and the raw QuTiP solver result.
- The generic extractor path supports two-mode and three-mode states through subsystem-aware partial traces.
- `reduced_cavity_state(...)` remains specific to the two-mode path. The three-mode path uses `reduced_storage_state(...)` and `reduced_readout_state(...)`.
- `measure_qubit(...)` acts on the reduced transmon state obtained from the runtime state convention. The exact probabilities are taken from the reduced density matrix before any optional confusion-matrix application or finite-shot sampling.
- Synthetic I/Q points remain available for lightweight sampling-only usage.
- The readout-chain path adds optional state-conditioned resonator traces, amplifier-noise I/Q sampling, and a reduced-state backaction model without changing the default no-chain behavior.

Direct tests: `tests/test_16_ideal_primitives_and_extractors.py`, `tests/test_19_fock_tomo.py`, `tests/test_26_state_prep_and_measurement.py`, `tests/test_27_three_mode_model.py`, `tests/test_28_thre`

22 Hilbert Space Truncation

- The two-mode model uses `n_cav` for storage truncation and `n_tr` for transmon truncation.
- The three-mode model uses `n_storage`, `n_readout`, and `n_tr`.
- The universal model layer also supports transmon-only and cavity-only special cases through the same subsystem-spec representation, while the wrapper models keep the historically used two-mode and three-mode constructor surfaces.
- The multilevel sideband and structured-transition helpers assume the selected ancilla levels lie within the chosen transmon truncation and raise if the requested transition falls outside $[0, \texttt{n_tr} - 1]$.

- State preparation helpers pad or truncate explicit amplitude vectors to the requested subsystem dimension.
- SQR angle arrays are still padded to the storage dimension in `cqed_sim/pulses/calibration.py`.
- The synthesis layer still distinguishes the full Hilbert dimension from restricted subspace dimension in `cqed_sim/unitary_synthesis/subspace.py`.

Direct tests: `tests/test_11_model_invariants.py`, `tests/test_24_displacement_selective_spectroscopy.p` `tests/test_26_state_prep_and_measurement.py`, `tests/test_27_three_mode_model.py`, `tests/unitary_synt`

23 Test Suite Audit

File	Targets, type, and main assertion	Status
`tests/test_01_sanity_and_free.py`	unit and numerical sanity checks for Hermiticity, free evolution, and Kerr phase accumulation in the two-mode runtime.	Passed
`tests/test_02_cavity_drive_and_kerr.py`	unit and regression checks for storage drive behavior and Kerr response.	Passed
`tests/test_03_dispersive_and_ramsey.py`	dispersive phase scaling and Ramsey pull in the two-mode model.	Passed
`tests/test_04_xy_phase_and_overlap.py`	XY phase convention and overlap cancellation for qubit pulses.	Passed
`tests/test_05_detuning_and_frames.py`	rotating-frame detuning signs and frame handling.	Passed
`tests/test_06_leakage_drag_and_higher_order.py`	multilevel leakage, DRAG behavior, and higher-order term smoke checks.	Passed
`tests/test_07_convergence_regression.py`	transition convergence and regression against a stored reference.	Passed
`tests/test_08_timeline_and_hardware.py`	waveform timeline assembly and basic hardware distortion semantics.	Passed
`tests/test_09_runtime_policy.py`	runtime-budget and fast-path regression checks.	Passed
`tests/test_10_chi_convention.py`	direct checks of runtime χ sign, transition-pull meaning, and observable defaults.	Passed
`tests/test_11_model_invariants.py`	model dimensions, invariants, and expectation consistency.	Passed
`tests/test_12_pulse_semantics.py`	pulse-area and pulse-shape semantics.	Passed
`tests/test_13_hardware_extended.py`	ZOH, low-pass, sideband, quantization, and crosstalk behavior.	Passed
`tests/test_14_dissipation.py`	explicit T_1, T_ϕ, and storage damping checks using collapse operators.	Passed
`tests/test_15_numerics_performance.py`	runtime, compile-cache, and runtime scaling smoke checks.	Passed
`tests/test_16_ideal_primitives_and_extractors.py`	ideal primitives/SNAP/SQR primitives and the two-mode extractor path.	Passed
`tests/test_17_gate_library_integration.py`	gate-library and integration path.	Passed
`tests/test_18_noise_and_sideband.py`	sideband exchange and combined noise behavior in the two-mode runtime.	Passed
`tests/test_19_fock_tomo.py`	reusable tomography and leakage-calibration helpers.	Passed
`tests/test_20_gaussian_iq_convention.py`	Gaussian I/Q axis, phase, and detuning sign convention.	Passed
`tests/test_21_qubox_convention_reconciliation.py`	reconciliation between runtime conventions and externally documented qubox-style conventions.	Passed
`tests/test_22_support_aware_objective.py`	support-aware objective wiring in synthesis-related helpers.	Passed
`tests/test_23_sqr_additive_amplitude_correction.py`	additive $d\lambda$ correction convention for SQR tones.	Passed

File	Targets, type, and main assertion	Status
`tests/test_24_displacement_selective_spectroscopy.py`	selective-spectroscopy frequencies and displacement conventions.	Passed
`tests/test_25_tensor_product_conventions.py`	conventions of tensor ordering, block indices, and active σ_z sign.	Passed
`tests/test_26_state_prep_and_measurement.py`	measurement ordering, mixed initial states, sampled qubit measurement, and the direct prepare-compile-simulate-measure workflow.	Passed
`tests/test_27_three_mode_model.py`	explicit basis-index, operator-embedding, partial-trace, analytic diagonal-energy, two-mode reduction, and dispersive shift checks for the three-mode model.	Passed
`tests/test_28_three_mode_drives_and_noise.py`	drives and noise, storage/readout ringdown, transmon T_1, transmon dephasing, positivity, and readout-response checks for the three-mode path.	Passed
`tests/analysis/test_spectrum.py`	vacuum-referenced eigenspectrum output, bare-state labeling, sideband level-spacing interpretation, and energy-level plotting smoke checks.	Passed
`tests/test_31_multilevel_sideband_extensions.py`	bare-transition sideband Hermiticity, $g \leftrightarrow f$ swap, detuned synchronization, shelving isolation, and multilevel-noise regression checks.	Passed
`tests/test_32_kerr_sign_notebook_regression.py`	notebook-scale Kerr evolution matches the documented self-Kerr sign rather than a flipped-sign surrogate.	Passed
`tests/test_33_usage_examples_spectroscopy.py`	fixed-Floquet propagation, spectroscopy regression plus guards that Tutorials 06 and 07 plot physical transition detuning instead of raw carrier.	Passed
`tests/test_34_universal_cqed_model.py`	model-model equivalence to the compatibility wrappers, multilevel anharmonicity, and cavity-only runtime support.	Passed
`tests/test_35_tutorial_api_conventions.py`	tutorial regression checks for the public Ramsey fit key `delta_omega` and explicit bare-frequency spectroscopy labels in the calibration notebooks.	Passed
`tests/test_58_floquet.py`	Floquet static-limit regression, exact cosine harmonic decomposition, propagator/quasienergy consistency, Sambe-route comparison, multiphoton resonance detection, and overlap-based branch tracking.	Passed
`tests/unitary_synthesis/test_subspace.py`	subspace basis ordering and round-trip embedding.	Passed
`tests/unitary_synthesis/test_metrics.py`	fidelity gauges and leakage metrics.	Passed
`tests/unitary_synthesis/test_primitives_and_backends.py`	primitives and backends, pulse backends.	Passed
`tests/unitary_synthesis/test_optimization.py`	optimization and time optimization.	Passed
`tests/unitary_synthesis/test_synthesis_and_targets.py`	synthesis and targets, and MPS-target behavior.	Passed with one optional skip
`tests/unitary_synthesis/test_free_evolve_cond_phase.py`	free-evolve cond-phase formulas.	Passed
`tests/unitary_synthesis/test_hardware_grid_parallel.py`	hardware, grid, and parallel-multistart behavior.	Passed
`tests/unitary_synthesis/test_progress_reporting.py`	progress reporting and serialization.	Passed
`tests/unitary_synthesis/test_reporting_and_determinism.py`	reporting and determinism.	Passed
`tests/unitary_synthesis/test_time_policy_and_conditional_phase.py`	time-policy and conditional-phase decomposition.	Passed
`examples/audits/tests/test_experiment_conventions_audit2.py`	experiment conventions audit, tensor ordering, waveform signs, and relative phases.	Passed
`examples/workflows/tests/test_sqt_workflow_artifacts.py`	sqt workflow artifact serialization and replay.	Passed

File	Targets, type, and main assertion	Status
`examples/studies/tests/test_snap_opt_convergence.py`	`snap_opt_convergence` regression for the SNAP optimization study.	Passed
`examples/studies/tests/test_snap_opt_achieves_zero_coherent_error.py`	`snap_opt_achieves_zero_coherent_error` above the empirical limit.	Passed
`examples/studies/tests/test_snap_opt_generalization_sweeps.py`	`snap_opt_generalization_sweeps` failure, runtime budget, and parameter sweeps.	Passed
`examples/studies/tests/test_snap_opt_preserves_low_frequency_content.py`	`snap_opt_preserves_low_frequency_content` produced.	Passed
`examples/paper_reproductions/tests/test_PRL-133_convergence_runtime.py`	`test_PRL-133_convergence_runtime` runtime-budget regression.	Passed
`examples/paper_reproductions/tests/test_PRL-133_cross-frequency.py`	`test_PRL-133_cross-frequency` test.	Passed
`examples/paper_reproductions/tests/test_PRL-133_optimization_limit.py`	`test_PRL-133_optimization_limit` regression.	Passed
`examples/paper_reproductions/tests/test_PRL-133_components_consistency.py`	`test_PRL-133_components_consistency`	Passed
`examples/paper_reproductions/tests/test_PRL-133_metrics_bounds.py`	`test_PRL-133_metrics_bounds` and fail-fast validation.	Passed
`examples/paper_reproductions/tests/test_PRL-133_metrics_regression.py`	`test_PRL-133_metrics_regression`	Passed
`examples/paper_reproductions/tests/test_PRL-133_unitarity_trace.py`	`test_PRL-133_unitarity_trace` unitarity and trace-preservation checks.	Passed
`examples/paper_reproductions/tests/test_PRL-133_reproduce_baseline.py`	`test_PRL-133_reproduce_baseline` reproduction smoke test.	Passed
`examples/smoke_tests/tests/test_sanity.py`	`test_sanity` partial-workflow sanity checks.	Passed
`examples/smoke_tests/tests/test_sqr_SQR_calibration.py`	`test_sqr_SQR_calibration` smoke and guarded-benchmark checks.	Passed
`examples/smoke_tests/tests/test_sqr_speed_limit_analytical.py`	`test_sqr_speed_limit_analytical` equivalence checks.	Passed

24 Test Execution Results

Commands used:

```
E:\Programs\python.exe -m pytest tests -q
E:\Programs\python.exe -m pytest examples/audits/tests examples/workflows/tests \
  examples/studies/tests examples/paper_reproductions/tests \
  examples/smoke_tests/tests -q
```

Observed results:

- `tests/`: 222 collected, 221 passed, 1 skipped, 0 failed.
- `examples/*/tests`: 49 collected, 49 passed, 0 failed.
- The single skip is the optional external MPS-reference check in `tests/unitary_synthesis/test_synthesis_an`
- Warning-only notes: two `numpy.trapz` deprecation warnings in `tests/test_12_pulse_semantics.py` and `tests/test_20_gaussian_iq_convention.py`, plus one QuTiP `sqrtm` singular-matrix warning in the open-system sideband fidelity regression.

Interpreter note: `E:\Program Files\Python311\python.exe` from `AGENTS.md` is not present in this workspace, so the suite was run with `E:\Programs\python.exe` (Python 3.11.8).

25 Verified Behaviors Backed by Tests

- Two-mode tensor ordering, basis labels, block indexing, and active Pauli/Bloch sign conventions are directly tested.

- The runtime meaning of χ as a projector-based conditional transition shift is directly tested, including the rule that negative χ lowers the transmon transition frequency with bosonic occupancy.
- The textbook-to-runtime projector translation is supported by direct σ_z sign tests and by runtime χ transition tests, even though the exact algebraic translation is not asserted by a single dedicated unit test.
- The three-mode tensor ordering, operator embeddings, partial traces, diagonal energies, and analytic dispersive shifts are directly tested.
- The three-mode solver path is directly tested for drive isolation, storage/readout-specific damping, transmon relaxation, transmon dephasing, positivity, and qubit-conditioned readout response.
- State preparation helpers are directly tested for ordering and mixed-state behavior.
- The qubit measurement abstraction is directly tested for exact probabilities, confusion-matrix application, finite-shot sampling, and synthetic I/Q output shape.
- The static-spectrum helper is directly tested for vacuum-referenced energies, bare-state labeling, and sideband-relevant energy differences.
- The direct prepare-compile-simulate-measure composition path is directly tested as the recommended library usage pattern.
- The moved example workflows still execute correctly in their new locations.

26 Gaps Between Code Conventions and Test Coverage

- The runtime-to-synthesis translation helper exposes synthesis coefficients aligned with the runtime convention, but the library still does not force every synthesis-side caller to route through that helper automatically.
- Higher-order χ_2, χ_3, and higher-order Kerr coefficients are covered indirectly, but not with isolated sign-and-factor analytic tests of the same style now used for the three-mode diagonal-energy checks.
- The readout-chain model is physically motivated, but it remains an effective resonator/filter/amplifier abstraction rather than a full microwave-network or POVM calibration workflow.
- The backend path is dense and piecewise constant; it is intended for parity and small-system acceleration checks, not as a drop-in replacement for every QuTiP workflow.

27 Potential Ambiguities

Moderate: the repository migrated on March 11, 2026 from an older $-\chi n_s n_q$ convention to the current $+\chi n_s n_q$ convention. Historical notes or saved plots created before that migration may still use the old sign.

Moderate: higher-order pull and Kerr parameters are falling-factorial coefficients, not ordinary polynomial coefficients.

Moderate: the runtime writes transmon physics in terms of n_q rather than $\sigma_z/2$, so ground-state energy offsets differ from some textbook presentations.

Moderate: `EnergySpectrum.energies` are vacuum-referenced in the selected frame, so rotating-frame spectra can contain negative excited-state energies even though the vacuum is fixed at zero.

Moderate: synthetic I/Q output is a convenience sampling layer, not a calibrated hardware-response model.

Low: the explicit complex-envelope sign $e^{+i(\omega t+\phi)}$ is repository-specific and must be preserved when adding pulse families.

Low: `FrameSpec` still stores the storage frame in the legacy field `omega_c_frame`; the `omega_s_frame` alias removes ambiguity but does not rename the underlying field.

Low: the same field name `chi` may still appear in both Hamiltonian-level dispersive contexts and effective readout-response contexts for compatibility, so the surrounding section must determine the intended interpretation.

28 Summary

`cqed_sim` is a reusable pulse-level cQED simulation library with a universal subsystem-based model core plus thin compatibility wrappers for the two-mode transmon–storage path and the three-mode storage–transmon–readout extension. The authoritative runtime conventions are:

- transmon-first tensor ordering,
- $|g\rangle = |0\rangle$, $|e\rangle = |1\rangle$, $|f\rangle = |2\rangle$,
- QuTiP-standard Pauli signs,
- internal Hamiltonians represented in rad/s, i.e. as $H/\hbar$,
- seconds for time,
- complex envelopes with $e^{+i(\omega t+\phi)}$ and therefore `carrier` $= -\omega_{\text{transition}}$ for resonant rotating-frame spectroscopy,
- dispersive physics written in terms of $n_q = b^\dagger b$ with $+\chi n_s n_q$-style conditional terms,
- and Lindblad collapse operators as implemented in `cqed_sim/sim/noise.py`.

Negative χ means that higher bosonic occupancy lowers the transmon transition frequency, and positive Kerr coefficients raise adjacent bosonic transition spacings. Higher-order nonlinear coefficients are stored in falling-factorial form. Runtime frequencies are effective dressed parameters of the chosen reduced model unless a section explicitly says otherwise. Vacuum-referenced spectral output is a reporting convention only and does not alter physical transition frequencies.

The experiment-facing layer has three measurement modes: a default lightweight exact-probability path, an opt-in physical readout-chain path with resonator response, Purcell filtering, amplifier noise, and reduced-state backaction, and an opt-in stochastic continuous-readout replay path built on monitored SME trajectories. The library also exposes operational strong-readout disturbance helpers for threshold studies, plus optional nonlinear and exchange Hamiltonian terms, aligned runtime/synthesis parameter translation helpers, calibration-target utilities, and an opt-in dense NumPy/JAX backend path.

29 Gate Library (`cqed_sim.gates`)

The `cqed_sim.gates` subpackage provides a structured library of ideal unitary gates organised by subsystem type. All functions return QuTiP `qt.Qobj` instances. The package-wide tensor-ordering and unit conventions (Section 29) apply throughout.

29.1 Single-Qubit Gates (`cqed_sim.gates.qubit`)

Basis: $|g\rangle = \mathrm{basis}(2,0)$, $|e\rangle = \mathrm{basis}(2,1)$. Rotation convention:

$$R_{\hat{n}}(\theta) = \exp\left(-i\frac{\theta}{2}\,\hat{n}\cdot\vec{\sigma}\right).$$

The most general equatorial rotation (`rphi`):

$$R_{\phi}(\theta) = \exp\left[-i\frac{\theta}{2}\,(\cos\phi\,X + \sin\phi\,Y)\right].$$

At $\phi = 0$ this equals $R_x(\theta)$; at $\phi = \pi/2$ it equals $R_y(\theta)$. This is the form generated by an on-resonance drive in the rotating frame.

Named Clifford and T gates are *exact ideal matrices*:

- X, Y, Z: standard Pauli matrices.
- Hadamard: $H = (X+Z)/\sqrt{2}$.
- Phase gate: $S = \mathrm{diag}(1, i)$; satisfies $S^2 = Z$.
- T gate: $T = \mathrm{diag}(1, e^{i\pi/4})$; satisfies $T^2 = S$.

29.2 Multilevel Transmon Transition-Selective Gates (`cqed_sim.gates.transmon`)

These gates act on a `dim`-dimensional transmon Hilbert space. The general transition-selective rotation between levels j and k ($0 \le j < k <$ `dim`) is:

$$R_{\phi}^{j,k}(\theta) = \exp\left[-i\frac{\theta}{2}\Big(e^{-i\phi}|j\rangle\langle k| + e^{i\phi}|k\rangle\langle j|\Big)\right].$$

All other levels receive the identity operator. This is an exact matrix exponential; no Hamiltonian truncation beyond the finite Hilbert space is applied.
Aliases:

- `r_ge`$(\theta, \phi, \mathrm{dim} = 3)$: $R_{\phi}^{0,1}(\theta)$. At $\mathrm{dim} = 2$ matches the qubit `rphi`.
- `r_ef`$(\theta, \phi, \mathrm{dim} = 3)$: $R_{\phi}^{1,2}(\theta)$. Requires $\mathrm{dim} \ge 3$.

29.3 Bosonic Cavity Gates (`cqed_sim.gates.bosonic`)

All bosonic gates are *exact ideal unitaries* within the finite Fock-space truncation of dimension `dim`.

Displacement.

$$D(\alpha) = \exp\Big(\alpha\, a^\dagger - \alpha^*\, a\Big)\,.$$

Implemented via `qt.displace`. For large $|\alpha|$ a larger `dim` is required to control truncation error.

Oscillator phase rotation.

$$R(\theta) = \exp(-i\theta\, a^\dagger a).$$

Acts as $|n\rangle \to e^{-in\theta}|n\rangle$.

Parity gate.

$$\Pi = \exp(i\pi\, a^\dagger a).$$

Acts as $|n\rangle \to (-1)^n|n\rangle$. Equals $R(-\pi)$.

Squeezing.

$$S(\zeta) = \exp\Big(\tfrac{1}{2}\zeta^*\, a^2 - \tfrac{1}{2}\zeta\, (a^\dagger)^2\Big)\,.$$

Implemented via `qt.squeeze`.

Self-Kerr evolution (unitary gate).

$$U_K(t) = \exp\left[-i\frac{K}{2}t\,\hat{n}(\hat{n}-1)\right] = \exp\left[-i\frac{K}{2}t\, a^{\dagger 2}a^2\right].$$

This is a diagonal unitary in the Fock basis. The phase accumulated by $|n\rangle$ is $\exp[-i(K/2)\,t\,n(n-1)]$, with K carrying the same sign as the Hamiltonian term $+K/2\, a^{\dagger 2}a^2$ used in the simulator (see Section on Hamiltonian Definitions). Note: `cqed_sim.sim.couplings.self_kerr` returns the Hamiltonian operator $K/2\, a^{\dagger 2}a^2$; this gate function returns the corresponding unitary gate.

SNAP gate.

$$S(\{\phi_n\}) = \sum_{n=0}^{N-1} e^{i\phi_n}|n\rangle\langle n|.$$

Accepts a sparse dictionary `{n: phase}` (unspecified levels receive zero phase) or a dense array.

29.4 Qubit-Cavity Conditional and Interaction Gates (`cqed_sim.gates.coupled`)

Tensor ordering: qubit first, cavity second, consistent with the rest of `cqed_sim`. The composite basis order for the default qubit_dim = 2 case is:

$$|g,0\rangle,\ |g,1\rangle,\ \ldots,\ |g,N-1\rangle,\ |e,0\rangle,\ |e,1\rangle,\ \ldots,\ |e,N-1\rangle.$$

Dispersive phase gate — two conventions. The `convention` parameter selects one of:

- `"n_e"` (default, matches `UniversalCQEDModel`):

$$H = \chi\, \hat{n}_{\text{cav}}\, |e\rangle\langle e|, \quad U(t) = |g\rangle\langle g| \otimes I + |e\rangle\langle e| \otimes e^{-i\chi t\hat{n}}.$$

  The $|g\rangle$ branch acquires no photon-number-dependent phase.

- `"z"` (Pauli-Z style):

$$H = \frac{\chi}{2}\, \hat{n}_{\text{cav}}\, Z, \quad U(t) = e^{-i\chi t\hat{n}/2} \otimes |g\rangle\langle g| + e^{+i\chi t\hat{n}/2} \otimes |e\rangle\langle e|.$$

The two conventions produce the same relative dispersive phase between the $|g\rangle$ and $|e\rangle$ branches, but differ by a branch-dependent photon-number phase. The `"n_e"` form matches the $+\chi\, n_c\, n_q$ Hamiltonian (with $n_q = |e\rangle\langle e|$) used in all model-level code.

Conditional cavity rotation.

$$U = |g\rangle\langle g| \otimes I + |e\rangle\langle e| \otimes e^{-i\theta\hat{n}} \quad (\texttt{control_state="e"}).$$

The `control_state` parameter flips control to $|g\rangle$ when set to `"g"`.

Conditional displacement.

$$CD(\alpha) = |g\rangle\langle g| \otimes D(+\alpha) + |e\rangle\langle e| \otimes D(-\alpha) \quad \text{(symmetric form)},$$
$$CD(\alpha_g, \alpha_e) = |g\rangle\langle g| \otimes D(\alpha_g) + |e\rangle\langle e| \otimes D(\alpha_e) \quad \text{(general form)}.$$

Controlled parity.

$$U = |g\rangle\langle g| \otimes I + |e\rangle\langle e| \otimes \Pi.$$

Controlled SNAP.

$$U = |g\rangle\langle g| \otimes I + |e\rangle\langle e| \otimes S(\{\phi_n\}).$$

Number-selective qubit rotation (SQR). `sqr` and `multi_sqr` use **qubit-first, cavity-second** tensor ordering, consistent with every other qubit-cavity gate in the package:

$$U_{\text{SQR}}(\theta, \phi; n) = R_\phi(\theta) \otimes |n\rangle\langle n| + I \otimes \sum_{m \neq n} |m\rangle\langle m|.$$

The generalised form:

$$U = \sum_n R_{\phi_n}(\theta_n) \otimes |n\rangle\langle n|.$$

The dense helper `sqr_op(thetas, phis)` in `cqed_sim.core.ideal_gates` also uses qubit-first, cavity-second ordering with a dense array interface.

Jaynes–Cummings and blue-sideband gates. Qubit first, cavity second. These are *Hamiltonian-generated* unitaries, computed via matrix exponentiation in the truncated space:

$$H_{\text{JC}} = g\Big(\sigma_+ \otimes a + \sigma_- \otimes a^\dagger\Big), \quad U_{\text{JC}}(t) = e^{-itH_{\text{JC}}},$$

$$H_{\text{blue}} = g\Big(\sigma_+ \otimes a^\dagger + \sigma_- \otimes a\Big), \quad U_{\text{blue}}(t) = e^{-itH_{\text{blue}}}.$$

Here $\sigma_+ = |e\rangle\langle g|$ and g is the vacuum Rabi coupling in rad/s.

Beam splitter (mode-mode exchange). Mode a first, mode b second:

$$H_{\mathrm{BS}} = g\Big(a^\dagger b + a\, b^\dagger\Big), \quad U_{\mathrm{BS}}(t) = e^{-itH_{\mathrm{BS}}}.$$

Equivalent to `beamsplitter_unitary`(dim_a, dim_b, $g \cdot t$).

29.5 Two-Qubit Gates (`cqed_sim.gates.two_qubit`)

Standard four-dimensional unitaries. Tensor ordering: **control first, target second**. Computational basis: $|gg\rangle, |ge\rangle, |eg\rangle, |ee\rangle$. All are *exact ideal matrices.*

- **CNOT**: $|g\rangle\langle g| \otimes I + |e\rangle\langle e| \otimes X$.
- **CZ**: $\mathrm{diag}(1, 1, 1, -1)$.
- **CP**(ϕ): $\mathrm{diag}(1, 1, 1, e^{i\phi})$. At $\phi = \pi$ equals CZ.
- **SWAP**: $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$.
- **iSWAP**: $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & i & 0 \\ 0 & i & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$.
- $\sqrt{\mathrm{iSWAP}}$: $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1/\sqrt{2} & i/\sqrt{2} & 0 \\ 0 & i/\sqrt{2} & 1/\sqrt{2} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$. Satisfies $(\sqrt{\mathrm{iSWAP}})^2 = \mathrm{iSWAP}$.

iSWAP and $\sqrt{\mathrm{iSWAP}}$ can be approximately generated by an exchange Hamiltonian $H = J(\sigma_+\sigma_- + \sigma_-\sigma_+)$; however, `iswap_gate()` and `sqrt_iswap_gate()` return the exact target matrices regardless of physical generation pathway and are therefore classified as *ideal unitaries* rather than *Hamiltonian-generated unitaries.*